

OSSP

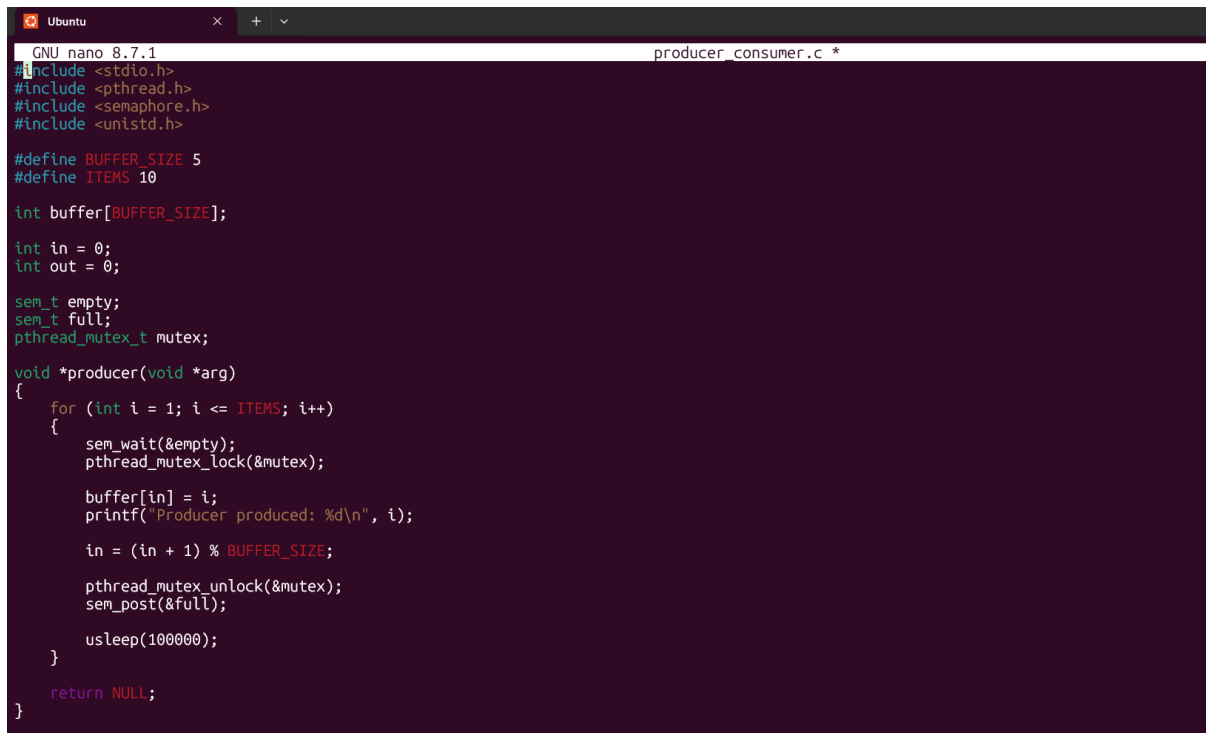
Practical Week - 12

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1-:

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.



```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Producer produced: %d\n", i);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        usleep(100000);
    }
    return NULL;
}
```

```
Ubuntu x + v
GNU nano 8.7.1 producer_consumer.c *
}
return NULL;
}
void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        usleep(150000);
    }
    return NULL;
}
int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was
```

```
Ubuntu x + v
GNU nano 8.7.1 producer_consumer.c *
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        usleep(150000);
    }
    return NULL;
}
int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Output:-

```
nocturnal@nocturnal:~$ nano producer_consumer.c
nocturnal@nocturnal:~$ gcc producer_consumer.c -o producer_consumer -pthread
nocturnal@nocturnal:~$ ./producer_consumer
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Producer produced: 8
Consumer consumed: 6
Producer produced: 9
Consumer consumed: 7
Producer produced: 10
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10
nocturnal@nocturnal:~$
```

Task-2:

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

GNU nano 8.7.1

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Acquired Resource 1\n");

    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Acquired Resource 2\n");

    printf("Thread 1: Working...\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Released resources\n");

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Acquired Resource 1\n");

    sleep(1);

    printf("Thread 2: Waiting for Resource 2\n");
```

```
void *thread2(void *arg)
{
    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Acquired Resource 1\n");

    sleep(1);

    printf("Thread 2: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Acquired Resource 2\n");

    printf("Thread 2: Working...\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Released resources\n");

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);
}
```

```
Ubuntu × + ▾
GNU nano 8.7.1 deadlock_prevention.c *

    sleep(1);

    printf("Thread 2: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Acquired Resource 2\n");

    printf("Thread 2: Working...\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Released resources\n");

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("No deadlock occurred.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
              ^C Location
              ^_ Go To Line
```

Output –

```
nocturnal@nocturnal:~$ nano deadlock_prevention.c
nocturnal@nocturnal:~$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
nocturnal@nocturnal:~$ ./deadlock_prevention
Thread 2: Waiting for Resource 1
Thread 2: Acquired Resource 1
Thread 1: Waiting for Resource 1
Thread 2: Waiting for Resource 2
Thread 2: Acquired Resource 2
Thread 2: Working...
Thread 2: Released resources
Thread 1: Acquired Resource 1
Thread 1: Waiting for Resource 2
Thread 1: Acquired Resource 2
Thread 1: Working...
Thread 1: Released resources
No deadlock occurred.
nocturnal@nocturnal:~$
```